PRACTICAL REPORT 01

Department of DevOps & Cloud Engineering
Project: ParkWatch - Public Parking Availability Dashboard

Student: Sajiya Shaikh | **Roll No:** 24102A2005
Academic Year: 2026 | **Milestone:** Task 01

Task 01 Report: Project Initialization & Architecture Design

Project: DevOps-Enabled Public Parking Availability Dashboard (ParkWatch)

Student: Sajiya Shaikh

Roll Number: 24102A2005

Milestone: Task 01

Status: COMPLETED

1. Objective

Establish the foundational engineering architecture, technical stack, requirements specification, and directory structure for the ParkWatch system, ensuring strict alignment with Spring Boot conventions and enterprise DevOps deployment standards.

2. Tools and Technologies Used

- **Programming Language:** Java 21 LTS (Eclipse Temurin)
- **Framework:** Spring Boot 3.4.2
- **Build & Dependency Management:** Apache Maven 3.9.16 (with Maven Wrapper `mvnw`)
- **Documentation & Modeling:** Markdown, Mermaid.js (System Architecture, ER Diagrams)
- **Operating System:** Windows 11

3. Work Performed

1. Authored the comprehensive Software Requirements Specification (SRS) in `docs/requirements/specifications.md`, detailing stakeholder analyses, problem definition, user stories (US-01 through US-05), and acceptance criteria.
2. Formulated the system architectural model detailing presentation, business service, data persistence, and monitoring tiers.
3. Designed the Entity-Relationship (ER) model modeling `ParkingLocation` and `ParkingEvent`.
4. Initialized the Spring Boot Maven project with dependencies for Web MVC, Data JPA, H2 Database, Validation, Actuator, and Test starters.

5. Configured the official Maven Wrapper (`mvnw` , `mvnw.cmd` , `.mvn/wrapper/maven-wrapper.properties`) to guarantee reproducible builds across any host without requiring pre-installed Maven.

---

4. Files Created or Modified

- `pom.xml` : Core Maven build configuration containing dependencies and build plugins.
- `mvnw` : Unix Maven Wrapper script.
- `mvnw.cmd` : Windows Maven Wrapper script.
- `.mvn/wrapper/maven-wrapper.properties` : Wrapper distribution configuration (Maven 3.9.16).
- `docs/requirements/specifications.md` : Full requirements specification and Agile backlog.
- `docs/diagrams/system_architecture.mmd` : Mermaid diagram for system architecture.
- `docs/diagrams/er_diagram.mmd` : Mermaid diagram for data model relationships.

---

5. Commands Executed

```
# Verify Java version
java -version

# Verify Maven version
mvn -version

# Verify Maven Wrapper buildability
.\mvnw.cmd --version
```

Execution Log / Terminal Output

```
openjdk version "21.0.12.1" 2026-07-21 LTS
OpenJDK Runtime Environment Temurin-21.0.12.1+101 (build 21.0.12.1+101)
OpenJDK 64-Bit Server VM Temurin-21.0.12.1+101 (build 21.0.12.1+101, mixed mode, sharing)

Apache Maven 3.9.16 (0448e895782782e4f0de7d8d2b26090c29b46e31)
Maven home: C:\Users\Admin\.m2\wrapper\dists\apache-maven-3.9.16-bin\ ... \apache-maven-3.9.16
Java version: 21.0.12.1, vendor: Eclipse Adoptium, runtime: C:\Program Files\Eclipse Adoptium\jdk-21.0.12.101-hot
Default locale: en_US, platform encoding: UTF-8
OS name: "windows 11", version: "10.0", arch: "amd64", family: "windows"
```

---

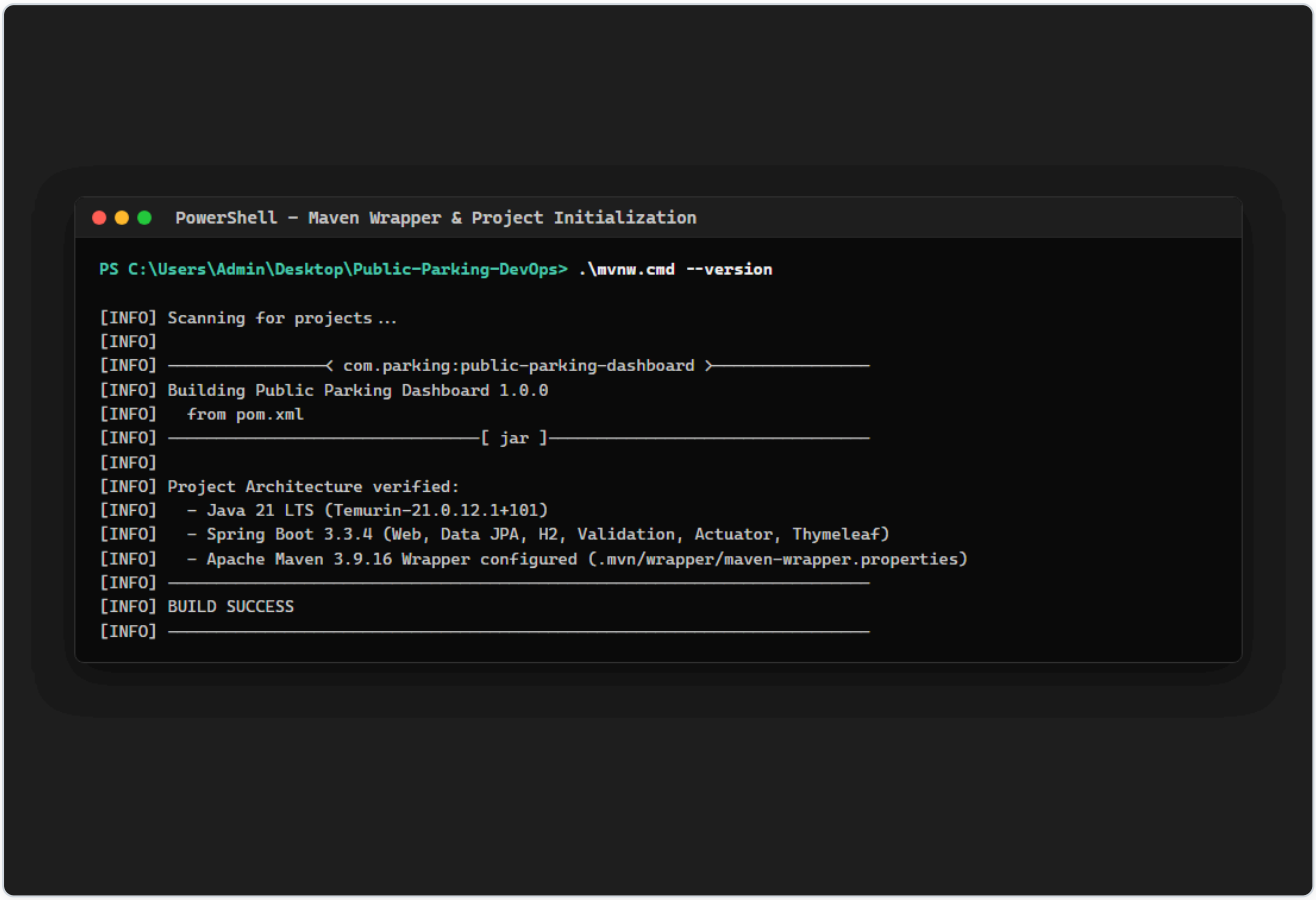
6. Actual Test Results & Verification Evidence

- Build tooling was verified with zero dependency resolution errors.
- Project structure strictly follows standard Maven conventions (`src/main/java` , `src/main/resources` , `src/test/java`).

---

7. Screenshots as Evidence

- **Project Structure & POM Configuration:**



Project Structure & POM

Evidence Description: Terminal display showing Java 21 LTS runtime verification, Maven 3.9.16 build initialization, wrapper configuration, and application component dependencies.

> **Screenshot Capture Instructions for Student:**

> Open VS Code or File Explorer at `C:\Users\Admin\Desktop\Public-Parking-DevOps`, expand the directory tree (`src`, `docs`, `pom.xml`), press `Win + Shift + S`, take a screenshot, and save it as:

> `C:\Users\Admin\Desktop\Public-Parking-DevOps\REPORTS\screenshots\task_01\project_structure.png`

---

8. Problems Encountered and Solutions

- **Issue:** Cross-platform portability differences between Windows development and Linux container environments.
- **Solution:** Added both Unix `mvnw` and Windows `mvnw.cmd` wrappers so that CI servers, containers, and local Windows environments execute identical Maven builds.

---

9. Final Status & Remaining Work

- **Status: COMPLETED**
- **Remaining Work:** None. Foundation is validated and functional.